

IMAGE PROCESSING

PROJECT

1. Applying watermark on an image

```
import os
from PIL import Image, ImageDraw, ImageFont
import matplotlib.pyplot as plt

def add_image_watermark(input_image, watermark_image, output_image, transparency):
    original = Image.open(input_image)
    watermark = Image.open(watermark_image)

    watermark = watermark.resize(original.size)

    watermark = watermark.convert("RGBA")
    watermark_with_transparency = Image.new("RGBA", watermark.size)

    for x in range(watermark.width):
        for y in range(watermark.height):
            r, g, b, a = watermark.getpixel((x, y))
            watermark_with_transparency.putpixel((x, y), (r, g, b, int(a * transparency)))

    watermarked = Image.alpha_composite(original.convert("RGBA"), watermark_with_transparency)

    watermarked = watermarked.convert("RGB")
    watermarked.save(output_image, "JPEG")

input_image = 'image.png'
watermark_image = 'watermark.png'
output_image = 'watermarked_image.jpg'
transparency = 0.6

add_image_watermark(input_image, watermark_image, output_image, transparency)

fig, axs = plt.subplots(1, 3, figsize=(15, 6))

original_img = Image.open(input_image)
axs[0].imshow(original_img)
axs[0].axis('off')
axs[0].set_title("Original Image")

watermark_img = Image.open(watermark_image)
axs[1].imshow(watermark_img)
axs[1].axis('off')
```

```

axs[1].set_title("Watermark Image")

watermarked_img = Image.open(output_image)
axs[2].imshow(watermarked_img)
axs[2].axis('off')
axs[2].set_title("Watermarked Image")

plt.show()

```



2. Customize Image and Text Watermarks to Photos with Batch Processing and Transformations

```

import os

from PIL import Image, ImageDraw, ImageFont

import matplotlib.pyplot as plt

def display_images(input_image_path, output_image_path, title=""):
    # Load images
    input_image = Image.open(input_image_path)
    output_image = Image.open(output_image_path)

    # Display images
    fig, axs = plt.subplots(1, 2, figsize=(12, 6))
    fig.suptitle(title, fontsize=16)

```

```
axs[0].imshow(input_image)
axs[0].axis('off')
axs[0].set_title("Input Image")
```

```
axs[1].imshow(output_image)
axs[1].axis('off')
axs[1].set_title("Output Image")
```

```
plt.show()
```

```
def watermark_images(mode, input_image=None, watermark_image=None, output_image=None,
text=None,
```

```
        input_folder=None, output_folder=None, transparency=0.6, position="center",
scale=0.3, font_path=None, font_size=30, color=(255, 255, 255)):
```

```
    if mode == "image":
```

```
        # Image watermarking
```

```
        if input_image and watermark_image and output_image:
```

```
            add_image_watermark(input_image, watermark_image, output_image, transparency,
position, scale)
```

```
            print(f"Image watermark applied and saved as {output_image}")
```

```
            display_images(input_image, output_image, title="Image Watermark")
```

```
    elif mode == "text":
```

```
        # Text watermarking
```

```
        if input_image and output_image and text:
```

```
            add_text_watermark(input_image, output_image, text, font_path, font_size, position, color,
transparency)
```

```
            print(f"Text watermark '{text}' applied and saved as {output_image}")
```

```
            display_images(input_image, output_image, title="Text Watermark")
```

```
    elif mode == "batch":
```

```
        # Batch processing with image watermark
```

```

if input_folder and watermark_image and output_folder:
    os.makedirs(output_folder, exist_ok=True)

    for filename in os.listdir(input_folder):
        if filename.lower().endswith(('.png', '.jpg', '.jpeg')):
            input_image_path = os.path.join(input_folder, filename)
            output_image_path = os.path.join(output_folder, f"watermarked_{filename}")
            add_image_watermark(input_image_path, watermark_image, output_image_path,
                                transparency, position, scale)
            print(f"Processed {filename}")

            display_images(input_image_path, output_image_path, title=f"Batch Image Watermark -
{filename}")

            print(f"Batch processing completed. Watermarked images saved in {output_folder}")

else:
    print("Invalid mode. Choose from 'image', 'text', or 'batch'.")

```

```

def add_image_watermark(input_image, watermark_image, output_image, transparency=0.6,
                        position="center", scale=0.3):

```

```

    original = Image.open(input_image).convert("RGBA")
    watermark = Image.open(watermark_image).convert("RGBA")

```

```

    # Resize watermark based on scale

```

```

    watermark = watermark.resize((int(original.width * scale), int(original.height * scale)),
                                Image.LANCZOS)

```

```

    # Set watermark transparency

```

```

    watermark_with_transparency = Image.new("RGBA", watermark.size)

```

```

    for x in range(watermark.width):

```

```

        for y in range(watermark.height):

```

```

            r, g, b, a = watermark.getpixel((x, y))

```

```

            watermark_with_transparency.putpixel((x, y), (r, g, b, int(a * transparency)))

```

```

# Calculate position based on specified location

if position == "center":
    x = (original.width - watermark.width) // 2
    y = (original.height - watermark.height) // 2
elif position == "bottom-right":
    x = original.width - watermark.width - 10
    y = original.height - watermark.height - 10
elif position == "top-left":
    x, y = 10, 10
elif position == "bottom-left":
    x = 10
    y = original.height - watermark.height - 10
elif position == "top-right":
    x = original.width - watermark.width - 10
    y = 10
else:
    x, y = 0, 0 # Default to top-left if position is unrecognized


# Apply watermark
watermarked_image = original.copy()
watermarked_image.paste(watermark_with_transparency, (x, y), watermark_with_transparency)
watermarked_image = watermarked_image.convert("RGB")
watermarked_image.save(output_image, "JPEG")


def add_text_watermark(input_image, output_image, text="Sample Watermark", font_path=None,
font_size=30, position="bottom-right", color=(255, 255, 255), transparency=0.6):
    original = Image.open(input_image).convert("RGBA")
    text_layer = Image.new("RGBA", original.size, (0, 0, 0, 0))

```

```
draw = ImageDraw.Draw(text_layer)

# Load font with specified size
if font_path:
    font = ImageFont.truetype(font_path, font_size)
else:
    # Use a built-in font with a specified size if no font path is provided
    font = ImageFont.truetype("arial.ttf", font_size)

# Calculate text size using bounding box for accurate placement
bbox = draw.textbbox((0, 0), text, font=font)
text_width, text_height = bbox[2] - bbox[0], bbox[3] - bbox[1]

# Calculate position
if position == "bottom-right":
    x = original.width - text_width - 25
    y = original.height - text_height - 25
elif position == "top-left":
    x, y = 10, 10
elif position == "center":
    x = (original.width - text_width) // 2
    y = (original.height - text_height) // 2
elif position == "bottom-left":
    x, y = 15, original.height - text_height - 25
elif position == "top-right":
    x = original.width - text_width - 10
    y = 10
else:
    x, y = 0, 0

# Draw text with transparency
```

```
draw.text((x, y), text, font=font, fill=color + (int(255 * transparency),))
```

```
watermarked_image = Image.alpha_composite(original, text_layer)
```

```
watermarked_image = watermarked_image.convert("RGB")
```

```
watermarked_image.save(output_image, "JPEG")
```

```
def batch_process_images(input_folder, watermark_image, output_folder, transparency=0.6,  
position="bottom-right", scale=0.3):
```

```
    os.makedirs(output_folder, exist_ok=True)
```

```
    for filename in os.listdir(input_folder):
```

```
        if filename.lower().endswith(('.png', '.jpg', '.jpeg')):
```

```
            input_image_path = os.path.join(input_folder, filename)
```

```
            output_image_path = os.path.join(output_folder, f"watermarked_{filename}")
```

```
            add_image_watermark(input_image_path, watermark_image, output_image_path,  
transparency, position, scale)
```

```
            print(f"Processed {filename}")
```

```
# Example Usage
```

```
print("Text watermark")
```

```
watermark_images(mode="text", input_image='image.png',  
output_image='text_watermarked_image.jpg', text="Confidential", position="bottom-right",  
font_size=20, transparency=0.3)
```

```
print("Image watermark")
```

```
watermark_images(mode="image", input_image='image.png', watermark_image='watermark.png',  
output_image='image_watermarked.jpg', transparency=0.6, position="center", scale=0.7)
```

```
print("Batch processing")
```

```
watermark_images(mode="batch", input_folder="images", watermark_image='watermark.png',  
output_folder="watermarked_images", transparency=0.5, position="top-left", scale=1)
```

Text Watermark

Input Image

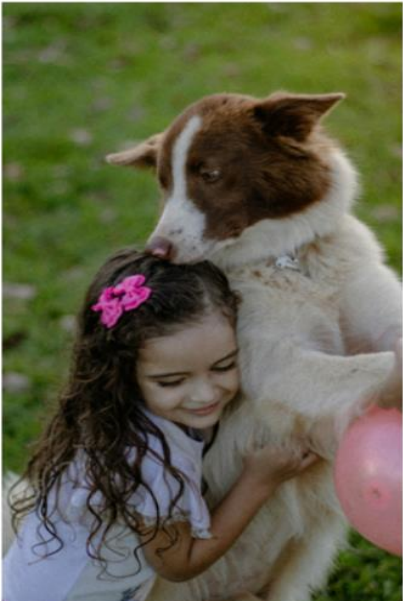


Output Image

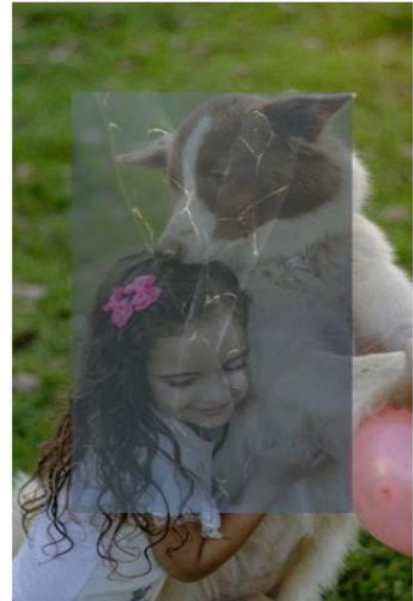


Image Watermark

Input Image



Output Image

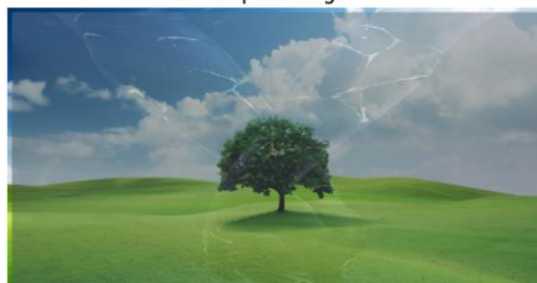


Batch Image Watermark - img1.jpg

Input Image

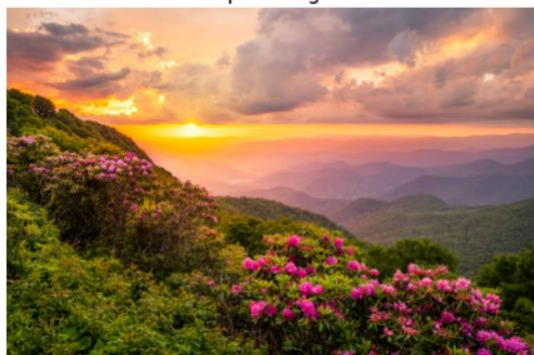


Output Image

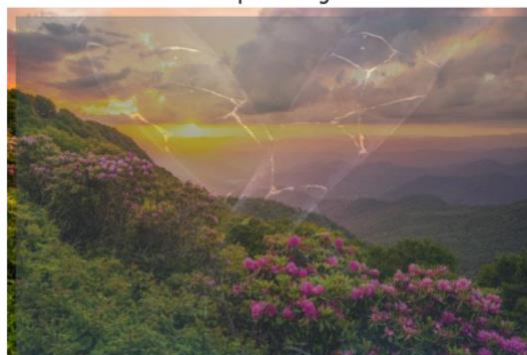


Batch Image Watermark - img2.jpg

Input Image



Output Image

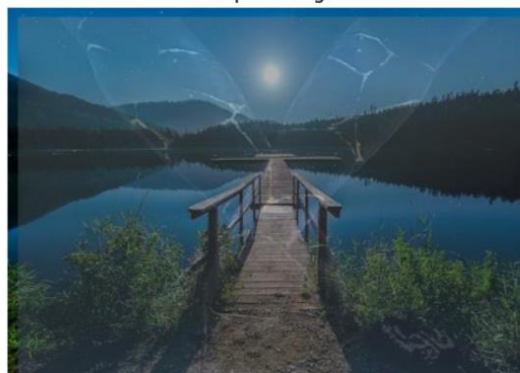


Batch Image Watermark - img3.jpeg

Input Image



Output Image



3. *Watermarking with Transformation*

```
import os
import numpy as np
from PIL import Image, ImageDraw, ImageFont, ImageOps
import matplotlib.pyplot as plt
import cv2

# 1. Grayscale Conversion and Thresholding
def apply_grayscale_and_threshold(image_path, threshold=128):
    image = Image.open(image_path).convert("L") # Convert to grayscale
    image = np.array(image)
    _, thresholded_image = cv2.threshold(image, threshold, 255, cv2.THRESH_BINARY)
    return Image.fromarray(thresholded_image)

# 2. Histogram Equalization
def apply_histogram_equalization(image_path):
    image = Image.open(image_path).convert("L") # Convert to grayscale
    image_np = np.array(image)
    equalized_image = cv2.equalizeHist(image_np)
    return Image.fromarray(equalized_image)

# 3. Image Smoothing and Sharpening
def apply_smoothing(image_path, kernel_size=5):
    image = Image.open(image_path)
    image_np = np.array(image)
    smoothed_image = cv2.GaussianBlur(image_np, (kernel_size, kernel_size), 0)
    return Image.fromarray(smoothed_image)

def apply_sharpening(image_path):
    image = Image.open(image_path)
    image_np = np.array(image)
    kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]]) # Simple sharpening kernel
    sharpened_image = cv2.filter2D(image_np, -1, kernel)
    return Image.fromarray(sharpened_image)

# 4. Sobel Edge Detection on Watermark for Outline Effect
def apply_sobel_edge_detection(image_path):
    image = Image.open(image_path).convert("L")
    image_np = np.array(image)
    sobel_x = cv2.Sobel(image_np, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(image_np, cv2.CV_64F, 0, 1, ksize=3)
    sobel_combined = cv2.magnitude(sobel_x, sobel_y)
    sobel_combined = np.uint8(sobel_combined) # Convert back to uint8
    return Image.fromarray(sobel_combined)

# Applying transformations to watermark
```

```

def add_watermark_with_transformations(input_image, watermark_image, output_image,
transformation="sharpen"):
    # Load images
    original = Image.open(input_image).convert("RGBA")
    watermark = Image.open(watermark_image).convert("RGBA")

    # Apply the selected transformation to the watermark
    if transformation == "grayscale_threshold":
        watermark = apply_grayscale_and_threshold(watermark_image)
    elif transformation == "histogram_equalization":
        watermark = apply_histogram_equalization(watermark_image)
    elif transformation == "smoothing":
        watermark = apply_smoothing(watermark_image)
    elif transformation == "sharpening":
        watermark = apply_sharpening(watermark_image)
    elif transformation == "sobel_edge":
        watermark = apply_sobel_edge_detection(watermark_image)

    # Convert watermark back to RGBA if it was modified
    if watermark.mode != "RGBA":
        watermark = watermark.convert("RGBA")

    # Ensure watermark matches the size of the original image
    watermark = watermark.resize(original.size, Image.LANCZOS)

    # Blend watermark with the original image at 50% transparency
    blended = Image.blend(original, watermark, alpha=0.5)
    blended = blended.convert("RGB") # Convert to RGB for saving as JPEG
    blended.save(output_image, "JPEG")
    return output_image

# Example Usage
input_image = 'image.png'
watermark_image = 'watermark.png'
output_image = 'watermarked_image.jpg'

# Apply watermark with transformations
add_watermark_with_transformations(input_image, watermark_image, output_image,
transformation="sharpening")

# Display results
fig, axs = plt.subplots(1, 3, figsize=(15, 6))
original_img = Image.open(input_image)
axs[0].imshow(original_img)
axs[0].axis('off')
axs[0].set_title("Original Image")

watermark_img = Image.open(watermark_image)

```

```
axs[1].imshow(watermark_img)
axs[1].axis('off')
axs[1].set_title("Watermark Image")
```

```
watermarked_img = Image.open(output_image)
axs[2].imshow(watermarked_img)
axs[2].axis('off')
axs[2].set_title("Watermarked Image with Transformation")
```

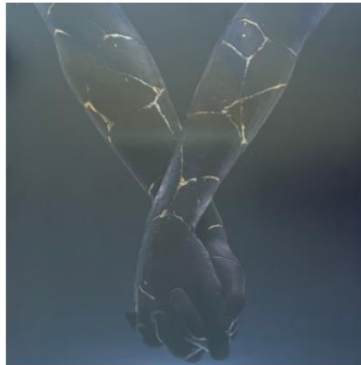
```
plt.show()
```

1. Grayscale Threshold

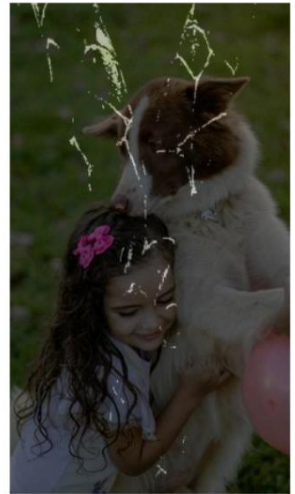
Original Image



Watermark Image

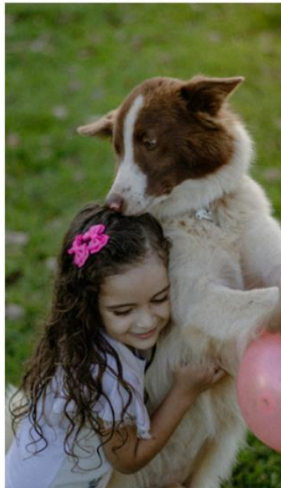


Watermarked Image with Transformation



2. Histogram Equalization

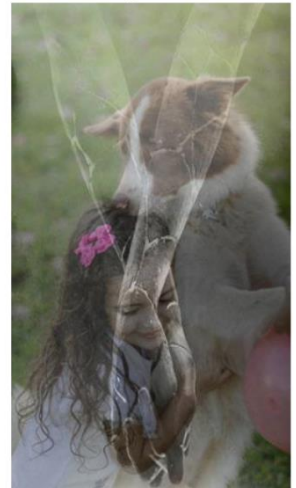
Original Image



Watermark Image

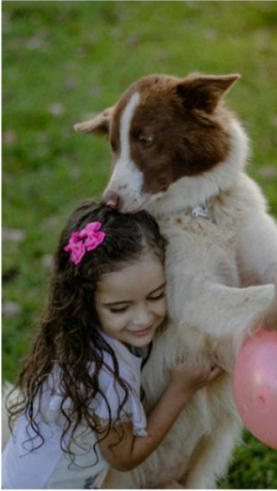


Watermarked Image with Transformation

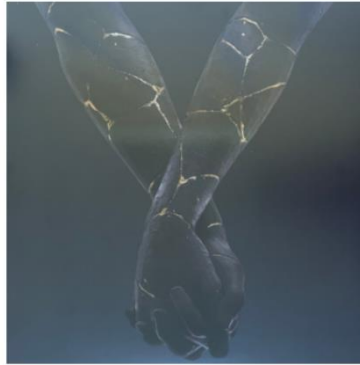


3. Smoothing

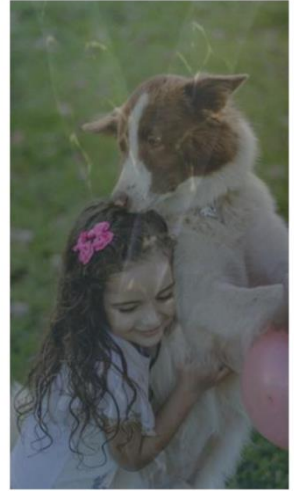
Original Image



Watermark Image



Watermarked Image with Transformation

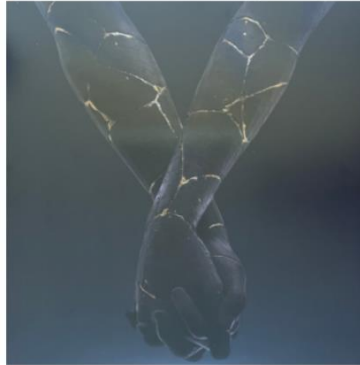


4. Sharpening

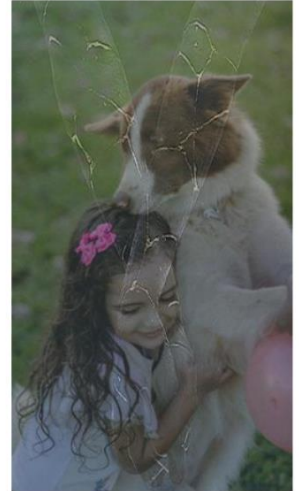
Original Image



Watermark Image

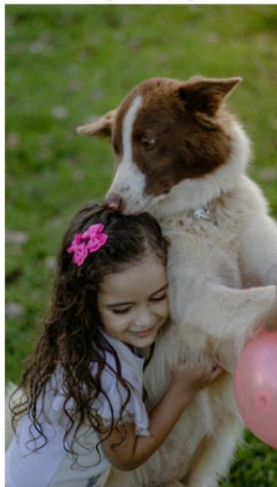


Watermarked Image with Transformation

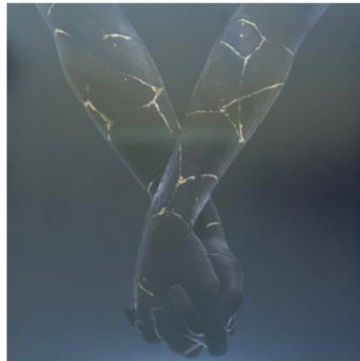


5. Sobel edge detection

Original Image



Watermark Image



Watermarked Image with Transformation

